

A Certain Precise Tilt of the Will

Programming for Beginners



“I assume that like any meaningful effort, the ritual involves sacrifice, the suppression of self-consciousness, and a certain precise tilt of the will, so that the will becomes transparent, hollow, a channel for the work.”

Annie Dillard
Teaching a Stone To Talk (1983)

Kevin Kirby
Northern Kentucky University
Spring 1998; Spring 2006

Contents

Introduction

- 1 Commenting, Quoting, Evaluating
- 2 Atoms and Lists
- 3 The Lambda: Forming Verbs
- 4 Flow of Control
- 5 Recursion
- 6 The Struct: Forming Nouns
- 7 Libraries and Design
- 8 The Lambda and the Aleph

Cover photo: Pyrometric cones, used since the Song dynasty in tenth century China to report the temperature of a kiln by their degree of softening; an analog computer. (From the kiln of Ana England.)

Introduction

Put your hand on the case of your computer. If it is warm, if it is expelling hot air, chances are that it is repeatedly reciting lines in an esoteric language. The recitation of these lines, called lines of *code*, have an effect on the parts of the computer, and have an effect on parts of the world. During one beat of your heart, your computer has recited a billion lines, lines composed of countless repeats and run through with common motifs. If this recitation is not *ritual*—a true ritual in the sense Annie Dillard uses in the epigraph to this book—what is it then?

A program is a complex incantation we humans write for our machines to recite. This book is about how to write incantations. Here is anthropologist Gary Witherspoon on the Navajo language:

The language of Navajo ritual is performative (Austin, 1962) not descriptive. Ritual language does not describe how things are; it determines how they will be. Ritual language is not impotent; it is powerful. It commands, compels, organizes, transforms, and restores. It disperses evil, reverses disorder, neutralizes pain, overcomes fear, eliminates illness, relieves anxiety, and restores order, health and well-being.¹

Programming languages are of great power, and of great delicacy. Like the Torah as portrayed in the Talmudic tradition, the omission of a single letter or the slightest jot could have catastrophic consequences in the world. Code errors have ravaged people with radiation and hurled their airliners down from the sky.

The languages of our machines lie below what we humans can really master. For example, the Pentium processor may be said to have its own language of ones and zeros, the on-and-off pulses that set other trains of on-and-off pulses in motion. In fact, is there a mediating language (“microcode”) between what we think of as the processor’s language and the actual control of electrical current flows on the processor chip. With such a mediating language, the language of the Pentium can be easily transliterated from its ones and zeros into a form that can be understood by human experts. Such transliteration looks like this:

```
mov    ah,0
int     16h
mov     dl,al
xlat    validchars
or      al,al
jz      getchar
mov     ah,2
int     21h
loop    getchar
```

¹ G. Witherspoon, *Language and Art in the Navajo Universe* (University of Michigan Press, 1977), p.34. The passage refers to the oft-cited work by J.L. Austin, *How To Do Things With Words*.

This is the most rudimentary poetry. A programmer should not be insensitive to the aesthetic nuances of code on the page. Here we have verbs followed by their objects, drawn from a sparse vocabulary. In effect, this code speaks like this

```
give      her, rice
say       look!
give      him, news
translate message
```

A processor will recite one of these little stanzas in a less than a nanosecond. When a programmer looks at passages of such code, presented in a “dump” from the core of the machine, and reads the lines at a rate of one per second, it is not unlike a scientist listening to dolphin speech, transposed down from ultrasound to a pitch we can hear. It is clear that this is not our native language.

Such machine code, when transliterated and tagged for humans, is called *assembly language*. The visible caesuras across the line of printed text are consonant with the sparseness of the language itself. The appeal of such an impoverished vocabulary is that it perfectly circumscribes one of the simplest useful models of computation: this is the simple world that the designers of a processor like the Pentium build for us. *Take this number from here. Add 1 to it. Put it over there. Is it zero? If so, repeat.* Such a narrow focus is so useful that even so-called *virtual machines*, entities that are not physical –are not etched in silicon– but exist only in the incantations of physical machines, are set up to speak in this rudimentary way.

Programmers almost always prefer to write code in a more enriched language. These higher level languages manage complexity, bend the computer to our will with extreme precision, and give us the ability to create our own vocabularies. When we speak in a higher level programming language we furnish worlds. Another way to put it is that we craft our own ontologies. Our programs construct and refer to new entities, abstract and concrete; we give them meaning.

The look of higher-level code is different from assembly code. Here is the language Python:

```
class Ceramic
    def __init__(self, f, a ):
        self.feldspar = f
        self.alumina = a

    def getAlumina( self ):
        return self.alumina

def main():
    x= Ceramic( 3, 5 )
```

The jagged edge left edge of the text is characteristic of modern source code, analogous to the indentation of the outline form used in a table of contents or by student term paper writers. In Python, in fact, the *whitespace* of indentation serves as punctuation.

A program, a piece of so-called *source code*, is a text. It is, in a sense we will later make quite precise, *static*. To make this text come alive, one needs an *interpreter* or a *compiler*. These are other programs that take source code (a few lines at a time, or entire compositions) and translate it into the language of the processor. For example, a program in Python may be translated into the machine language of the Pentium. Using a more elaborate but very common approach, a program written in the Java language will be translated into the language of a *virtual machine*; this virtual machine is a program that has already been translated into the language of the Pentium. So, unlike a poem, the meaning of a piece of source code is determinate; it is transferred to the execution of operations on a piece of hardware. (This hermeneutic deferral is called “operational semantics.”) What the flickering ones and zeros of the processor actually *mean* depends subtly on how it coheres within itself and how it corresponds to the world. This philosophical subtlety is perhaps *the* philosophical subtlety of computer science.

Many programs transform data. Although data, like code, is ultimately a kind of text, if it is represented in a well-defined language at all the language will not be a programming language. XML, the extensible markup language, is a language for structuring data as labeled hierarchical text. Here is an example:

```
<?xml version="1.0"?>
<!-- An example: The dog bit my hand -->
<sentence mood="indicative">
  <subject person="3" number="singular" animation="non-human">
    <form>
      <nounphrase>
        <article>the</article>
        <noun number="singular">dog</noun>
      </nounphrase>
    </form>
  </subject>
  <predicate>
    <form>
      <verbphrase>
        <verb tense="past" root="bite">bit</verb>
        <object>
          <form>
            <nounphrase>
              <adjective atype="possessive" person="1" number="singular">my</adjective>
              <noun number="singular">hand</noun>
            </nounphrase>
          </form>
        </object>
      </verbphrase>
    </form>
  </predicate>
</sentence>
```

If code and data are both textual objects, why not treat them the same way? A processor like the Pentium is an example of a model of computing called the *von Neumann machine*, named after the mathematician John von Neumann. In this model, a program (something that *processes*) is stored the same way as data (something that is *processed*). One higher level language in which source code and data may be treated on an equal footing is Scheme. Here is a transliteration of the XML example above in the Scheme language:

```

(sentence
; An example: the dog bit my hand
  (mood indicative)
  (subject (attributes (person 3) singular animate (not human))
    (form (noun-phrase
      (article definite (form "the"))
      (noun singular (form "dog")))))
  (predicate (attributes (tense past) (person 3) singular)
    (form (verb-phrase)
      (verb "bite" (form "bit"))
      (object (form (nounphrase
        (adjective (possessive (person 1) singular (form "my")))
        (noun singular (form "hand"))))))))

```

As we shall see, words like “sentence” and “predicate” here need not be mere labels; they can be commands with transformatory power. The text is a description for its own transformation, yielding another text that transforms itself, almost like a protein catalyzing its own metabolism, or a snake swallowing its tail.

Scheme is the particular language we will study in this book. It is a direct descendent of one of the two ancestral computer languages: Lisp, dating from the late 1950s. The second of the two ancestral languages is Fortran. In the loud acronyms of archaic computer books they were LISP and FORTRAN, for LISt Processing and FORMula TRANslation.

The descendents of Fortran have evolved into languages like C++, Java, and Python; their resemblance to their great ancestor has become attenuated as they have mutated into sleek beasts adapted to sundry technological niches.

By contrast, Scheme is only a mild streamlining of Lisp, looking pretty much like the 1950s model, but, one might say, without the fins. Its appeal is not the appeal of retro, however. In computer science time, the half-century since the birth of Lisp spans the rise and fall of empires, and to look at Scheme code is not so much to hear echoes of the Sputnik era, but to hear the bass notes of the *Ursprache*.

But even Lisp is only an overtone of a deeper note struck in pre-linguistic 1936: the lambda calculus. The λ marks the disembodied soul of computer science, its imaginary eigenvalue. It is on the excavated lessons of 1936 concerning the limitations of what we can compute that this book, in its last chapters, finally comes to rest. This is where programming ends and computer science begins.

Programming is not computer science. Indeed, one fundamental area of computer science deals with animating and interpreting natural systems (such as those that operate by the enigmatic principles of quantum mechanics, or the baroque dynamics of DNA processing). In these areas the notion of *program* is vexed.

Despite the emphasis above on how code animates a machine, it should also be kept in mind that code is designed to be read by humans. Much of the evolution of programming has been driven by the need for humans to write, read, and change code more efficiently. The irony of the discipline of programming is that a piece of code can be cut as precisely as a brilliant diamond, yet the economic pressure of the software business requires the

production of massive quantities of industrial code crafted to low standards and rarely inspected.

Such considerations would move us into the world of software engineering. This, however, is a book on programming. Programming is a liberal art. Programming is metamagic. The programmer is a meta-magus. To be deaf and dumb to the gigahertz hum of code, in all the digital devices that surround us, is a condition not worthy of the learned.
